Swift Conventions & Best Practices

Swift's official API Design Guidelines open with one goal: clarity at the point of use. Every convention below serves it. When one of them seems fussy, weigh it against how the code will read to the next person who opens the file.

1. Formatting & Whitespace

Indentation

Use spaces, not tabs. Xcode's default is 4 spaces.

Braces: Opening Brace on the Same Line

The opening brace goes at the end of the declaration line, preceded by one space:

```
// Good
if pikachu.isCharged {
    thunderbolt()
}

// Bad
if pikachu.isCharged
{
    thunderbolt()
}
```

`else` / `else if` sits on the same line as the closing brace:

```
if catchRate > 90 {
    ball = .poke
} else if catchRate > 80 {
    ball = .great
} else {
    ball = .ultra
}
```

Empty blocks may be written compactly as `{}`.

Spaces Around Colons

Bind the colon to the thing on its left: no space before, one space after.

```
// Good
let level: Int = 0
var badges: [String: Int] = [:]
struct Pokemon: Equatable { }

// Bad
let level : Int = 0
var badges : [String:Int] = [ : ]
```

Spaces Around Operators and Commas

One space on each side of binary operators and `->`. One space after a comma, never before.

```
// Good
let damage = basePower + criticalBonus
let party = ["Finn", "Jake", "BMO"]
func swampArea(width: Double, depth: Double) -> Double

// Bad
let damage = basePower+criticalBonus
let party = ["Finn" ,"Jake" , "BMO"]
func swampArea(width: Double,depth: Double)->Double
```

Exceptions, no spaces around these:

```
jake.name             // member access dot
0..<party.count       // range operators
1...151
```

No Semicolons

Swift does not use `;` to end statements. A semicolon is *legal* after any statement, and only *required* to put two statements on one line. Do neither.

```
let onionLayers = 7         // Good
let onionLayers = 7;        // Bad
```

No Parentheses Around Conditions

```
if isOgre {               // Good
while badges < 8 {        // Good
```

```
if (isOgre) {            // Bad, a C habit
```

Line Length and Wrapping

Keep lines under ~100 characters. When a comma-separated list is too long, put it all on one line or one item per line, never half and half:

```
// Good, fits
let trainer = Trainer(name: "Ash", town: "Pallet")

// Good, doesn't fit, so fully vertical
let trainer = Trainer(
    name: "Ash Ketchum",
    hometown: "Pallet Town",
    role: .champion,
)

// Bad, mixed
let trainer = Trainer(name: "Ash Ketchum",
    hometown: "Pallet Town", role: .champion)
```

When each element sits on its own line, add a trailing comma. The next diff becomes one line instead of two. Swift has always allowed trailing commas in array and dictionary literals; as of Swift 6.1 they work in argument lists and other comma-separated lists too.

Blank Lines

One blank line between functions and between logical groups of statements. Not two, not zero. No blank line right after an opening brace or before a closing brace.

One Statement Per Line

```
// Good
let hearts = 3
let swords = 2

// Bad
let hearts = 3; let swords = 2
```

Comments

Use `//` for ordinary comments, with one space after the slashes. Use `///` for documentation comments. Skip `/* */` block comments.

```swift
// Good: explains why
// The Pokédex API reports weight in hectograms, the UI shows kilograms.
let kilograms = Double(hectograms) / 10

// Bad: restates the code
// Divide hectograms by 10
let kilograms = Double(hectograms) / 10
```

---

2. Naming

Casing

Thing	Case	Example
Types, protocols, enums	`UpperCamelCase`	`struct GymBadge`, `protocol Rideable`
Variables, constants, functions	`lowerCamelCase`	`let candyCount`, `func loadPokedex()`
Enum cases	`lowerCamelCase`	`case inBattle`
Generic type parameters	`UpperCamelCase`, short	`<T>`, `<Element>`, `<Key, Value>`

Note the third row: Swift enum cases are lowercase, unlike most other languages.

Swift has no `SCREAMING_SNAKE_CASE`. Constants use `lowerCamelCase` like everything else:

```swift
let maximumPartySize = 6      // Good
let MAX_PARTY_SIZE = 6        // Bad, a C/Java/Python habit
```

No underscores in names, and no type or scope prefixes (`strName`, `m_count`, `NSFoo`). Swift gives you namespaces and a type system instead.

Acronyms Are All-One-Case

Uppercase them all or lowercase them all, matching whatever the name would otherwise start with:

```swift
let urlSession = URLSession.shared    // Good
let htmlBody = "…"                    // Good
```

```
struct HTTPRequest { }                  // Good

let uRLSession = …                      // Bad
let HtmlBody = …                        // Bad
```

Be Clear, Not Brief

Swift does not reward brevity. Include the words a reader needs and cut the ones they don't.

```
// Good
let remainingPokeBalls = 3
func removeBadge(at index: Int)

// Bad, too abbreviated
let remPB = 3
func rmBdg(i: Int)
```

But do drop words that add nothing at the call site:

```
// Bad, "Element" tells the reader what they can already see
finnsSwords.removeElement(rootSword)

// Good
finnsSwords.remove(rootSword)
```

Short names work when the scope is tiny and the meaning is plain: `i` in a small loop, `x` / `y` for coordinates.

Name Things by Their Role, Not Their Type

```
var battleCry: String       // Good, says what it's for
var string: String          // Bad, restates the type

var tournamentDeadline: Date    // Good
var dateVar: Date               // Bad
```

Booleans Read as Assertions

Prefix with `is`, `has`, `should`, `can`, or use a verb that reads as a claim about the subject:

```
var isEvolving: Bool         // Good
var hasGymBadge: Bool        // Good
var canFlee: Bool          // Good

var evolving: Bool           // Weaker
var flag: Bool               // Bad
```

Collections Are Plural, Single Values Are Singular

```
let hero = "Finn"                       // Good
let heroes = ["Finn", "Jake"]           // Good
let heroList = ["Finn", "Jake"]         // Redundant, the "es" already
says it
```

---

3. Constants & Variables

`let` first, `var` only when you must

Default to `let`. Switch to `var` when you reassign. The compiler then catches stray mutation, and a reader can see at a glance what changes.

```
// Good
let swampAcreage = 12.5
var donkeyComplaints = 0

// Bad, never reassigned, so it should be `let`
var swampAcreage = 12.5
```

Xcode flags these with "never mutated; consider changing to let". Fix those warnings.

Let Type Inference Do Its Job

Annotate the type when it isn't clear from the right-hand side, or when you need a type other than the default.

```
// Good, clear from the literal
let princess = "Fiona"
let kantoDexCount = 151
let catchRate = 0.75
```

```
// Redundant
let princess: String = "Fiona"
let kantoDexCount: Int = 151

// Good, the annotation is doing real work
let critChance: Float = 0.0625        // otherwise Double
let badges: [String: Int] = [:]       // empty literal has no inferable
type
var chosenStarter: Int?               // starts nil, type can't be
inferred
```

Use `_` for Values You Don't Need

```
let (_, defense) = statSpread
for _ in 1...3 {
    print("Mathmatical")
}
```

---

4. Strings

Interpolate, don't concatenate:

```
// Good
let message = "Get out of my swamp, \(name)! You and your \(count)
friends."

// Bad
let message = "Get out of my swamp, " + name + "! You and your " +
String(count) + " friends."
```

Check for emptiness with `isEmpty`, not by comparing to `""` or counting characters:

```
if ogreName.isEmpty { }          // Good
if ogreName == "" { }            // Bad
if ogreName.count == 0 { }       // Bad, and O(n) for strings
```

Multiline strings use `"""` delimiters, and the closing delimiter's indentation sets the baseline:

```
let query = """
    SELECT name, species
    FROM duloc_residents
    WHERE evicted = 1
    """
```

---

5. Control Flow

`guard` For Early Exit

When you validate a precondition, `guard` puts the requirement and the bail-out up front and leaves the happy path unindented. Every `guard` must exit its scope (`return`, `throw`, `break`, `continue`).

```
// Good
func recruit(hero: Hero?, for quest: Quest) {
    guard let hero else { return }
    guard quest.hasOpenSlots else { return }

    quest.add(hero)
}

// Bad, the real work is buried
func recruit(hero: Hero?, for quest: Quest) {
    if let hero = hero {
        if quest.hasOpenSlots {
            quest.add(hero)
        }
    }
}
```

Prefer `for-in` over Index Arithmetic

```
// Good
for hero in party {
    print(hero.name)
}

// Good, when you need the index too
for (index, hero) in party.enumerated() {
```

```
    print("\(index + 1). \(hero.name)")
}

// Bad
for i in 0..<party.count {
    print(party[i].name)
}
```

`switch` Over Long `if` / `else if` Chains

The compiler checks a `switch` over an enum for exhaustiveness, so omit `default` once you cover every case. Add a case later and the compiler points at every spot that needs handling. A `default` throws that away.

```
// Good
switch dungeonState {
case .exploring: rollEncounter()
case .victory: openTreasure()
case .defeat: respawnAtCamp()
}
```

Keep Nesting Shallow

Three levels of indentation is a signal to extract a function or invert a condition with `guard`.

---

6. Optionals

Beginners misuse optionals more than any other Swift feature, so these conventions carry more weight than the formatting ones.

Never Force-Unwrap to "Make It Compile"

```
let starter = chosenStarter!       // Bad, crashes when nil
```

`!` claims this can never be nil. If you can't prove that, don't write it.

Unwrap with `if let` Or `guard let`

Modern Swift lets you skip the repeated name (shorthand syntax):

```
// Good
if let rival {
    challenge(rival)
}

// Also fine, and required when renaming
if let rival = league.reigningChampion {
    challenge(rival)
}
```

Use `??` for a Fallback Value

```
let displayName = nickname ?? "Detective Pikachu"          // Good

let displayName = nickname != nil ? nickname! : "Detective Pikachu"
// Bad
```

Use Optional Chaining Instead of Nested Checks

```
let town = trainer.hometown?.name                    // Good
let partySize = trainer.party?.count ?? 0            // Good
```

Avoid Implicitly Unwrapped Optionals (`String!`)

They read like normal values and crash like force-unwraps. Beginner code has no good reason for one.

---

7. Collections

Declare with a literal when you know the contents, and with an annotated empty literal when you don't:

```
let starters = ["Bulbasaur", "Charmander", "Squirtle"]   // inferred
[String]
var badges: [String: Int] = [:]                          // annotation
needed
var nicknames: [String] = []                             // preferred
over [String]()
```

Use `isEmpty`, not `count == 0`. Use `first`/`last`, which return optionals, rather than `[0]`, which crashes on an empty collection.

```
if let lead = party.first { }      // Good
let lead = party[0]                // Bad, crashes if empty
```

Reach for `map` / `filter` / `reduce` when you transform a whole collection. They name *what* you're doing, and the result can be a `let`:

```
// Good
let names = party.map { $0.name }
let healthy = party.filter { $0.hp > 0 }
let totalExperience = battleRewards.reduce(0, +)

// More verbose, more mutable state
var names: [String] = []
for member in party {
    names.append(member.name)
}
```

Shorthand `$0` works for one-liners. Once the closure grows past a line or two, name the parameter.

---

8. Functions

Name by Whether There Are Side Effects

- **Does something?** Imperative verb: `sort()`, `append(_:)`, `rematch()`
- **Returns something without changing anything?** Noun phrase: `distance(to:)`, `sortedNames`

Swift pairs these up as verb / past participle:

```
party.sort()                  // mutates in place
let ranked = party.sorted()   // returns a new value
```

Factory-style methods start with `make`: `makeIterator()`.

Argument Labels Should Make the Call Site Read like English

You are designing the call site. Read it out loud.

```
// Good, reads as a phrase
party.insert(jake, at: 0)
league.register(ash, on: openingDay)
view.dismiss(animated: true)

// Bad, labels repeat the function name or vanish where you need them
party.insertMember(member: jake, index: 0)
league.register(ash, openingDay)      // what's the second thing?
```

Two specific rules from the official guidelines:

When the first argument forms a prepositional phrase, start the label at the preposition:

```
func removeBadge(at index: Int)         // trainer.removeBadge(at: 3)
func distance(to gym: Gym)              // pallet.distance(to: pewter)
```

Otherwise, fold the descriptive word into the function name and drop the first label:

```
func addSubview(_ view: View)           // container.addSubview(header)
```

Drop the label when the argument is not meaningfully distinct, as with value conversions and peer arguments:

```
func min(_ x: Int, _ y: Int) -> Int     // min(3, 7)
Int("151")                              // no label on a conversion
```

Separate External and Internal Names When It Helps Both Sides

The first name is what callers write; the second is what you use in the body.

```
func greet(_ person: String, from hometown: String) -> String {
    "Hello \(person)! Glad you could visit from \(hometown)."
}

greet("Fiona", from: "Duloc")
```

Give Parameters Defaults Instead of Writing Overloads

Defaults cut the number of functions a reader has to learn. Put the parameters you expect callers to default toward the end.

```
// Good
func fetchSubscribers(on plan: Plan, includeChurned: Bool = false) →
[Subscriber]

fetchSubscribers(on: mintUnlimited)
fetchSubscribers(on: mintUnlimited, includeChurned: true)
```

Omit `→ Void` and Empty Parentheses in Return Types

```
func rematch() { }              // Good
func rematch() → Void { }       // Bad, redundant
func rematch() → () { }         // Bad
```

Single-Expression Functions Can Omit `return`

```
// Good
func damage(power: Double, multiplier: Double) → Double {
    power * multiplier
}

// Also fine and never wrong, so pick one and hold to it within a file
func damage(power: Double, multiplier: Double) → Double {
    return power * multiplier
}
```

Keep Functions Short and Single-Purpose

A rough heuristic: if you can't describe what a function does in one sentence without "and," it does two things. Same for parameters. Past three or four, pass a struct instead.

Wrap Long Signatures One Parameter Per Line

```
func sign(
    player: Player,
    to club: Club,
    startingOn date: Date,
    notifyManager: Bool = true
) → Contract {
    // …
}
```

Note where the closing paren and brace go: paren on its own line at the original indentation, brace after it.

Document With `///`

Documentation comments use triple slashes and show up in Xcode's Quick Help (⌥-click). Start with a one-sentence summary, then tag the parameters and the return value.

```
/// Calculates the total damage a move deals across a multi-hit turn.
///
/// - Parameters:
///   - hits: The base power of each hit that landed.
///   - multipliers: The type effectiveness for each hit; one per hit.
/// - Returns: The total damage dealt, in hit points.
func totalDamage(hits: [Double], multipliers: [Double]) -> Double {
    zip(hits, multipliers)
        .map { $0.0 * $0.1 }
        .reduce(0, +)
}
```

Document every function another person will call. Don't document `getName()` with "gets the name."

Quick Checklist

Formatting

- [ ] Consistent indentation, spaces not tabs
- [ ] Opening braces on the same line
- [ ] No space before a colon, one space after
- [ ] Spaces around binary operators; space after commas only
- [ ] No semicolons, no parens around conditions
- [ ] Lines under ~100 chars; wrapped lists fully vertical

Naming

- [ ] `UpperCamelCase` types, `lowerCamelCase` everything else including enum cases and constants
- [ ] Acronyms all-one-case
- [ ] Clear over short; needless words dropped
- [ ] Named by role, not type
- [ ] Booleans read as assertions (`isEnabled`)

Declarations

- [ ] `let` unless you reassign
- [ ] Type annotations only where they earn their place
- [ ] `Int` and `Double` by default
- [ ] String interpolation, not concatenation

Control flow & optionals

- [ ] `guard` for early exits; happy path unindented
- [ ] `for-in` over index loops
- [ ] Exhaustive `switch` on enums, no `default`
- [ ] Zero force-unwraps (`!`)
- [ ] `??` for fallbacks, `?.` for chaining
- [ ] `isEmpty` over `count == 0`

Functions

- [ ] Verb names for side effects, noun names for returns
- [ ] Call site reads as an English phrase
- [ ] Labels start at the preposition, or fold into the name
- [ ] Defaults instead of overloads, defaulted params last
- [ ] No `-> Void`
- [ ] One job per function
- [ ] `///` docs on anything others call